

You're working with a **linear function** and a **loss function**, so let's go step by step to compute the derivatives with respect to W_1 and b .

1. Define Your Function and Loss

Your function is:

$$y = W_1 x_1 + b$$

Your loss function (assuming $L = \hat{y} - y$, which resembles a basic residual/error):

$$L = \hat{y} - y$$

where:

- $\hat{y}$ is the predicted value.
- y is the actual (target) value.

2. Compute the Derivatives

We want to compute **partial derivatives** of L with respect to W_1 and b .

Derivative w.r.t W_1 :

$$\frac{\partial L}{\partial W_1} = \frac{\partial}{\partial W_1} (\hat{y} - y)$$

Since $\hat{y} = W_1 x_1 + b$, we differentiate:

$$\frac{\partial L}{\partial W_1} = \frac{\partial}{\partial W_1} (W_1 x_1 + b - y)$$

$$\frac{\partial L}{\partial W_1} = x_1$$

Derivative w.r.t b :

$$\frac{\partial L}{\partial b} = \frac{\partial}{\partial b}(\hat{y} - y)$$
$$\frac{\partial L}{\partial b} = 1$$

3. Compute Example Values for Your Input

Given the **input tensor in PyTorch**:

```
import torch

X = torch.tensor([
    [10.0], # 10°C
    [38.0], # 38°C
    [100.0], # 100°C
    [150.0] # 150°C
])
```

For each x_1 , we can compute the derivatives:

x_1	$\frac{\partial L}{\partial W_1}$	$\frac{\partial L}{\partial b}$
10.0	10.0	1
38.0	38.0	1
100.0	100.0	1
150.0	150.0	1

4. Interpretation

- The derivative with respect to W_1 depends on x_1 , meaning the **steepness of the weight adjustment** is proportional to the input value.
- The derivative with respect to b is always **1**, meaning the bias affects the loss equally for all inputs.

If this is for **gradient descent**, you'd use these gradients to update W_1 and b :

$$W_1 = W_1 - \alpha \frac{\partial L}{\partial W_1}$$

$$b = b - \alpha \frac{\partial L}{\partial b}$$

where α is the learning rate.

Would you like me to show this in PyTorch code with actual values? 🚀

Yes! The symbols might look different, but the core **differentiation rules** from our **Differential Calculus refresher** still apply. Let's break it down and map it to the concepts we discussed earlier.

Mapping to Differential Calculus Concepts

In your equation:

$$y = W_1 x_1 + b$$

This is a **linear function** in terms of W_1 and b .

We want to find how the **loss function** $L = \hat{y} - y$ changes with respect to **each parameter** W_1

and b , which means we need to compute **partial derivatives**.

1. Partial Derivatives & Power Rule

When differentiating functions with multiple variables, we use **partial derivatives** (denoted as $\frac{\partial}{\partial W_1}$ instead of $\frac{d}{dx}$).

Using the **power rule** from calculus:

$$\frac{d}{dx}(ax) = a$$

we differentiate:

$$\frac{\partial}{\partial W_1}(W_1 x_1) = x_1$$

and since b is a constant:

$$\frac{\partial}{\partial W_1}(b) = 0$$

which gives:

$$\frac{\partial L}{\partial W_1} = x_1$$

Similarly, differentiating with respect to b :

$$\frac{\partial}{\partial b}(W_1 x_1 + b) = 1$$

which gives:

$$\frac{\partial L}{\partial b} = 1$$

2. Connection to the "Refresher on Differential Calculus"

Let's compare to our previous examples:

1. **We used the power rule:** $\frac{d}{dx}(x^n) = nx^{n-1}$
 - Here, W_1x_1 behaves like ax , where $a = W_1$, and its derivative is just x_1 .
 - Similarly, the derivative of a constant (b) is always **1**.
2. **We applied the sum/difference rule:**
 - The derivative of a sum is just the sum of the derivatives.
3. **We used the concept of rates of change:**
 - Here, the gradient tells us **how much** W_1 and b should be updated to minimize loss.

Why Does It Look Different?

The difference is in **notation**:

- In calculus, we usually write functions in terms of x and differentiate as $\frac{d}{dx}$.
- In machine learning, we use **parameters** (W_1, b) and write **partial derivatives** $\frac{\partial}{\partial W_1}$.

But the differentiation rules remain **exactly the same**!

Final Summary

- **Concepts from Differential Calculus apply directly:** We used the **power rule**, **sum rule**, and **constant rule** to differentiate.
- **Notation is slightly different:** Instead of $\frac{d}{dx}$, we use **partial derivatives** $\frac{\partial}{\partial W_1}$.
- **Same logic applies in neural networks:** Gradients are computed using these differentiation rules to optimize W_1 and b .

Would you like a **PyTorch implementation** to compute these derivatives? 